

สำหรับทีมพัฒนา ULMs — อ่านก่อนตัดสินใจว่าจะต่อ frontend เข้ากับ backend ด้วยวิธีไหน

เราห่างจากจุดที่ ต่อกันได้จริงแค่ไหน

เดินตามเส้นทางข้อมูล 5 สถานี · ทุกจุดที่ spike แก่จากโค้ดจริง
และจุดผิดพลาดที่ต้องแก้ก่อน เรียงตามความรุนแรง

วันที่ 10 สิงหาคม 2569 มีการทดลองเอา frontend/ ของทีมทั้งก้อน ไป
ต่อกับ backend ที่เขียนขึ้นจริง แล้วล็อกอินสำเร็จทั้งสามบทบาท ผ่านการ
ตรวจ 13 ข้อที่ขึ้นข้อมูล และ 15 ข้อในเบราว์เซอร์จริง

เอกสารนี้ไม่ได้เขียนเพื่อฉลองว่าสำเร็จ แต่เขียนเพื่อตอบว่า ระหว่างทาง
ต้องแก่อะไรบ้าง และ อะไรที่ยังพังอยู่ในโค้ดปัจจุบัน โดยที่ยังไม่มีใครเห็น
เพราะข้อมูลจำลองปิดบังไว้

จุดสำคัญที่สุดไม่ได้อยู่ที่ frontend หรือ backend แต่อยู่ที่ฐานข้อมูล —
ภาค 6

วิธีอ่านเอกสารนี้

เอกสารนี้เดินทางตามเส้นทางที่ข้อมูลเดินจริง ไม่ได้เรียงตามหัวข้อทฤษฎี เริ่มจากนิ้วที่กดปุ่มบนหน้าจอ แล้วไล่ตามไปจนถึงแถวในฐานข้อมูล ผ่านทั้งหมด 5 สถานี

เหตุผลที่เรียงแบบนี้: ปัญหาที่เจอเวลาต่อ frontend กับ backend มักไม่ได้อยู่ที่ปลายทางใดปลายทางหนึ่ง แต่อยู่ที่รอยต่อ ซึ่งจะมองไม่เห็นเลยถ้าอ่านแยกเป็น “บทของ frontend” กับ “บทของ backend”

ทุกสถานีตอบสามคำถามเดิมเสมอ

1. ทีมมีอะไรตอนนี้ — อ้างไฟล์และบรรทัดจริงในรีโป
2. spike เปลี่ยนอะไร — โค้ดก่อน/หลังของจริง
3. ยังพังตรงไหน — สิ่ง que spike ไม่ได้แก้ พร้อมระดับความรุนแรง

ระดับความรุนแรง

P0	แก้ก่อนใครทำงานต่อได้ ถ้าปล่อยไว้ งานที่เพิ่มจะต้องรื้อ หรือระบบไม่ปลอดภัย
P1	แก้ก่อนเริ่มลงมือทำหน้าที่มีฟอร์ม ต้นทุนโตขึ้นทุกสัปดาห์ที่เลื่อน
P2	แก้ตอนแตะโค้ดส่วนนั้นพอดี ไม่ต้องหยุดงานอื่นเพื่อมาแก้
P3	งานเก็บกวาด ทำตอนไหนก็ได้

กล่องสีแบบ

กล่องฟ้า	ใจความที่ต้องจำ ประโยคที่ร้อยหลายภาคเข้าด้วยกัน
กล่องส้ม	กับดัก หรือข้อสมมติที่ถ้าลืมแล้วข้อสรุปจะผิด
กล่องเขียว	สิ่งที่ทดสอบกับของจริงแล้ว มีผลการยืนยัน
กล่องแดง	วิธีที่ผิดจริง ๆ อยู่ในโค้ดตอนนี้และต้องเอาออก

สีประจำชั้นของระบบ

สีเดียวกันนี้ใช้ทั้งในเนื้อความ ไดอะแกรม และตารางตลอดเล่ม เห็นสีแล้วควรรู้ทันทีที่กำลังพูดถึงชั้นไหน

frontend	สิ่งที่รันในเบราว์เซอร์ผู้ใช้
สัญญา	ข้อตกลงเรื่องรูปร่างข้อมูล สิ่งที่เกิดทางระหว่างสองฝั่ง
เครือข่าย	เครือข่าย คุณก็ CORS สิ่งที่อยู่ระหว่างทาง
backend	สิ่งที่รันบนเซิร์ฟเวอร์ รวมถึงฐานข้อมูล

ข้อจำกัดของเอกสารนี้ที่ควรรู้อีก

spike ที่อ้างถึงต่อเฉพาะระบบล็อกอินเท่านั้น ไม่ใช่ทั้งระบบ เพราะเป็นเส้นเดียวที่ทั้งสองฝั่งมีของจริงพร้อมกัน ณ วันที่ทดลอง

ข้อสรุปเรื่อง “ต่อกันได้” จึงหมายถึง “กลไกการต่อใช้ได้จริง” ไม่ได้แปลว่าทั้งระบบพร้อมต่อ ส่วนที่เหลืออีก 15 procedure ยังไม่ได้เขียน

สารบัญ

วิธีอ่านเอกสารนี้	1
1 ภาพรวม — ทีมอยู่ตรงไหน spike อยู่ตรงไหน	5
1 สามคำถามที่เอกสารนี้ตอบ	5
2 แผนที่ 5 สถานี	5
3 ระยะห่างระหว่างทีมกับ spike	6
4 สิ่งที่น่าสนใจแล้วจริง ๆ	6
2 สถานีที่ 1 — หน้าจอและฟอร์ม	8
5 ทีมมีอะไรตอนนี้	8
6 จุดที่ 1 — หน้าล็อกอินเดาบบทบาทเอง	8
7 จุดที่ 2 — บทบาทเก็บอยู่ใน localStorage	9
8 จุดที่ 3 — รหัสผ่านทดสอบอยู่ในรีโป	10
9 จุดที่ 4 — ฟอร์มต้องกลายเป็น async	10
10 จุดที่ 5 — แถบข้างแสดงชื่อที่ตายตัว	11
11 สรุปสถานีที่ 1	11
3 สถานีที่ 2 — สัญญาและ type	12
12 สัญญาเดินทางมาแบบไหน	12
13 จุดที่ 6 — zod คนละรุ่น	12
13.1 ขาวดี: ยกตอนนี้ไม่พังเลย	13
14 จุดที่ 7 — type ของผู้ใช้ที่เขียนด้วยมือ	13

14.1 วิธีที่ spike ใช้กันไม่ให้เกิดอีก	13
15 จุดที่ 8 — รหัสข้อผิดพลาดคนละชุด	14
16 จุดที่ 9 — สัญญาเก่าค้างได้ และ CI มองไม่เห็น	14
17 สรุปสถานที่ 2	15
4 สถานที่ 3 — เครือข่ายและ session	16
18 ทีมเตรียมอะไรไว้แล้ว	16
19 กลไกที่เกิดขึ้นจริงเมื่อล็อกอิน	16
20 จุดที่ 10 — CORS ที่ยังไม่มี	16
21 จุดที่ 11 — proxy ที่ชี้ผิดที่	17
22 จุดที่ 12 — แยก “ต่อไม่ติด” ออกจาก “รหัสผิด” ไม่ได้	18
23 สรุปสถานที่ 3	18
5 สถานที่ 4 — server	19
24 ทีมมีอะไรตอนนี้	19
25 จุดที่ 13 — แปดไฟล์ที่ต้องเขียนเพิ่ม	19
26 บทเรียนจากโค้ดจริง — การกันการเดาบัญชี	20
27 สรุปสถานที่ 4	20
6 สถานที่ 5 — ฐานข้อมูล	21
28 สภาพปัจจุบัน	21
29 PO จุดที่ 14 — คอลัมน์ที่ใช้ล็อกอินไม่มี @unique	21
30 PO จุดที่ 15 — ไม่มีตารางเก็บ session	22

31 P1 จุดที่ 16 — จำนวนวันที่ยืมได้ ไม่ได้ขึ้นกับเครดิตอย่างเดียว	22
32 P1 จุดที่ 17 — “สังกัด” ของผู้ใช้เป็นการเดา	23
33 P2 จุดที่ 18 — ชื่อระดับเครดิตเป็นค่าที่ไม่บังคับ	23
34 P2 จุดที่ 19 — RoLeInfo เป็นตาราง ไม่ใช่ enum	24
35 สรุปสถานที่ 5	24
 7 ต้องทำอะไรบ้าง และเรียงลำดับอย่างไร	25
36 ตารางรวมทั้ง 19 จุด	25
37 ลำดับที่ต้องลงมือ	25
38 สิ่งที่ยังไม่รู้	26
 8 ภาคผนวก	27
ภาคผนวก ก — ทุกไฟล์ที่ spike แก่	27
ภาคผนวก ข — วิธีรัน spike เพื่อดูด้วยตัวเอง	28
ภาคผนวก ค — ผลการทดสอบ 28 ข้อ	28
ชั้นข้อมูล 13 ข้อ	28
ชั้นหน้าจอ 15 ข้อ	29
ภาคผนวก ง — ที่มาของข้อเท็จจริงทุกข้อ	29

ภาค 1

ภาพรวม — ทีมอยู่ตรงไหน spike อยู่ตรงไหน

1 สามคำถามที่เอกสารนี้ตอบ

คำถามที่ทำให้เกิดเอกสารนี้มีสามข้อ และทั้งสามข้อตอบด้วยข้อมูลจากรีโปจริง ไม่ใช่การประเมิน

1. spike แกะอะไรจากโค้ดจริงบ้าง — ตอบครบทุกไฟล์ในภาคผนวก ก
2. ทีมห่างจากจุดที่ต่อกันได้จริงแค่ไหน — ตอบในหัวข้อ 3
3. จุดผิดพลาดสำคัญที่ต้องแก้ก่อนคืออะไร — ตอบตลอดเล่ม สรุปในภาค 7

ข้อสรุปสามบรรทัด ถ้าอ่านต่อไม่ไหว

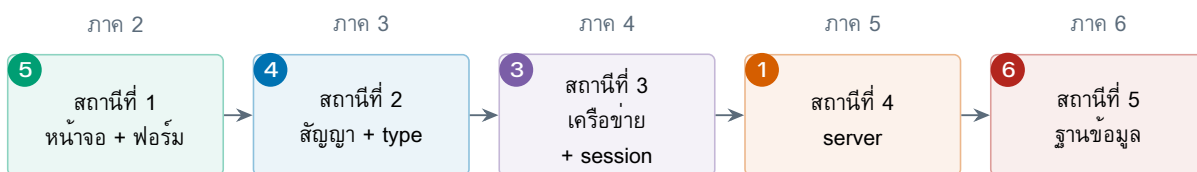
หนึ่ง กลไกการต่อใช้ได้จริง พิสูจน์แล้วด้วยการล็อกอินสำเร็จทั้งสามบทบาท ผ่านการตรวจ 28 ข้อ (13 ที่ขึ้นข้อมูล + 15 ในเบราร์เซอร์)

สอง ราคาที่ต้องจ่ายก่อนเริ่มคือยก zod จาก 3 เป็น 4 ตอนนี้นียกแล้วไม่พังเลย แต่ต้นทุนจะโตตามจำนวนฟอร์มที่ยังไม่ได้ทำ

สาม ปัญหาที่หนักที่สุดไม่ได้อยู่ที่โค้ดฝั่งไหนเลย แต่อยู่ที่ **ฐานข้อมูล** — ไม่มี @unique บนคอลัมน์ที่ใช้ล็อกอิน และไม่มีตารางเก็บ session ทั้งสองข้อต้องเขียน migration ซึ่งแปลว่า ต้องตกลงกันทั้งทีมก่อน

2 แผนที่ 5 สถานี

รูปที่ 1 คือโครงของทั้งเล่ม แต่ละกล่องคือหนึ่งภาค ตัวเลขในวงกลมคือจำนวนจุดที่ต้องแก้ที่พบในสถานีนั้น



ตัวเลขในวงกลม = จำนวนจุดที่ต้องแก้ที่พบในสถานีนั้น รวมทั้งหมด 19 จุด

สถานีที่ 5 เป็นสีแดงเพราะเป็นสถานีเดียวที่แก้แล้วกระทบทุกคนในทีมพร้อมกัน — ทุกจุดในนั้นต้องเขียน migration หรือต้องตกลงกันก่อน

รูปที่ 1: เส้นทางที่ข้อมูลเดินจริงเมื่อผู้ใช้กดปุ่มเข้าสู่ระบบ และภาคที่อธิบายแต่ละสถานี

3 ระยะห่างระหว่างที่มิกกับ spike

ตารางที่ 1 คือคำตอบตรง ๆ ของคำถาม “ห่างแค่ไหน” ตัวเลขทุกช่องนับจากไฟล์จริงในรีโป

ตารางที่ 1: สภาพของแต่ละส่วน ณ วันที่ 10 สิงหาคม 2569

ส่วน	ทีมมีอะไรตอนนี้	spike ต้องเพิ่มอะไร
frontend โครง	โครงสร้างหมด routing, RBAC, AppShell, ui-kit, i18n, theme	ไม่ต้องแตะเลย — สวมกับสัญญาได้พอดี
frontend หน้าจอ	เสร็จจริง 8 จาก 26 หน้า (login + admin 6 หน้า + 1)	ไม่เกี่ยว — spike ต่อเฉพาะ auth
frontend ข้อมูล	จำลองทั้งหมด	เปลี่ยน 3 เส้นเป็นของจริง (login/me/logout)
สัญญา สัญญา	ยังไม่มี	ไฟล์ที่ generate มา 56 บรรทัด
สัญญา zod	^3.23.8	ต้องยกเป็น 4.4.3
เครือข่าย คุณก็	api-client.ts เตรียม credentials: include ไว้แล้ว	ต้องตั้ง CORS ให้ระบุ origin ตรง ๆ
backend auth	ไม่มีเลย — src/ มีแต่โครง Nest เปล่ากับ Prisma	8 ไฟล์ใหม่
backend ฐานข้อมูล	33 model 1 migration	ต้องมี migration เพิ่ม — spike เลี่ยงไปก่อนโดยไม่แก้

อ่านตารางนี้แล้วภาพที่ควรได้คือ: ฝั่ง frontend ไกลมาก ฝั่ง backend ไกลมาก และฐานข้อมูลคือที่ยังไม่มีใครแตะ

ตัวเลขที่ควรจำจากตารางนี้

frontend ของทีมต้องแก้แค่ 14 ไฟล์ เพิ่ม 4 ลบ 1 ซึ่งน้อยกว่าที่ทุกคนกลัวกันมาก แต่ backend ต้องเขียนใหม่ 8 ไฟล์ จากศูนย์ และฐานข้อมูลต้องมี migration ที่ยังไม่มีใครเขียน

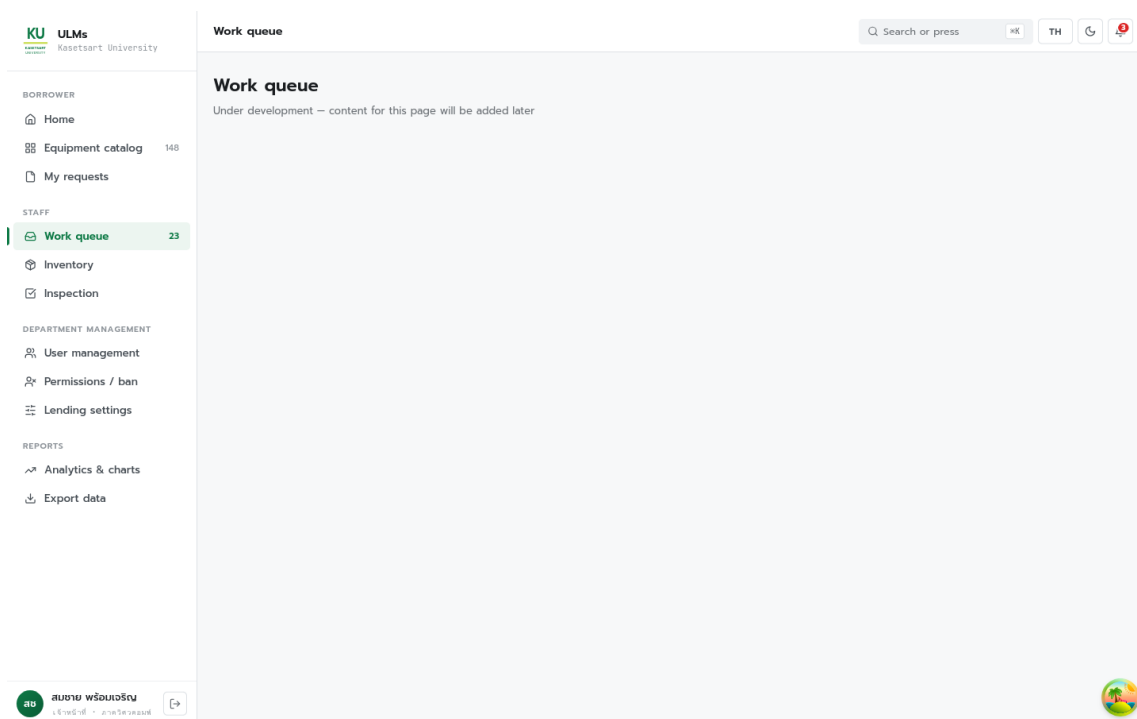
4 สิ่งที่พิสูจน์แล้วจริง ๆ

ผลการทดสอบ 28 ข้อ ผ่านหมด

ขั้นข้อมูล 13 ข้อ (spike-option-b/frontend-real/e2e-check.mjs) ใช้ @trpc/client ตัวเดียวกับที่แอปใช้ — ยังไม่ล็อกอินแล้วถาม auth.me, ล็อกอินด้วยบัญชีภายใน, คุณก็ติดไปกับคำขอถัดไป, รหัสผ่านผิด, อีเมลไม่ใช่ KU, ข้อความยาวเกิน max(200), อาจารย์ล็อกอินด้วยอีเมล KU, นิสิตเครดิตต่ำได้ D3, logout แล้วคุณก็หาย

ขั้นหน้าจอ 15 ข้อ (spike-option-b/browser-check/ui-check.mjs) เปิด Firefox จริง กรอกฟอร์มจริง — รวมสามข้อที่ทดสอบด้วยวิธีอื่นไม่ได้เลย คือ คุณก็เป็น httpOnly จริง (document.cookie อ่านไม่เจอ), กด F5 แล้ว session ไม่หลุด, และ RBAC ยึดจริงเมื่อพิมพ์ URL ตรง ๆ

รูปที่ 2 คือภาพหน้าจอจริงหลังล็อกอินสำเร็จด้วยบัญชี `test_staff` สังเกตว่าเนื้อหาเขียนว่า “Under development” — ถูกต้องแล้ว เพราะหน้านั้นยังเป็น placeholder สิ่งทีุ่รูปนี้พิสูจน์คือกรอบรอบ ๆ ทั้งหมดมาจากเซิร์ฟเวอร์จริง ไม่ใช่ข้อมูลจำลอง



รูปที่ 2: หน้าจอหลังล็อกอินด้วยบัญชีจริงจากฐานข้อมูล เมนูฝั่งซ้ายมาจากบทบาทที่เซิร์ฟเวอร์ตอบกลับ ไม่ใช่บทบาทที่เบราว์เซอร์เลือกเอง

ภาค 2

สถานีที่ 1 — หน้าจอและฟอร์ม

สถานีแรกคือสิ่งที่ผู้ใช้เห็นและแตะ ขาวดีของสถานีนี้คือ**โครงทั้งหมดใช้ได้เลย** ขาวร้ายคือ ตรรกะการตัดสินใจว่า “ใครเป็นใคร” ถูกเขียนไว้ผิดที่

5 ทีมมีอะไรตอนนี้

โครงพื้นฐานเสร็จหมดและไม่ต้องแตะเลยตอนต่อ backend ได้แก่ routing พร้อม RBAC guard, AppShell (แถบข้าง/แถบบน), ชุด ui-kit, i18n ไทย-อังกฤษ, สลับธีมสว่าง/มืด และ design tokens

ส่วนเนื้อหาจอเสร็จไม่เท่ากันมาก ตามตารางที่ 2

ตารางที่ 2: สถานะหน้าจอทั้ง 26 หน้า นับจากไฟล์จริงใน frontend/src/features/

กลุ่ม	เสร็จ	รายละเอียด
auth	1/1	login-page.tsx 124 บรรทัด สองวิธีเข้าระบบ
admin	6/7	dashboard, users (511 บรรทัด), status, audit, config, reports — ขาด settings
borrower	0/7	placeholder 6 บรรทัดทั้งหมด
staff	0/7	placeholder 6 บรรทัดทั้งหมด
supervisor	0/2	placeholder
reports	0/2	placeholder
รวม	8/26	หน้าที่เสร็จทุกหน้ากินข้อมูลจาก features/admin/mock-data.ts

ตัวเลข 8/26 อ่านผิดได้ง่าย

อย่าอ่านว่า “เสร็จ 31%” เพราะงานที่เหลือไม่ได้เท่ากันทุกหน้า
หน้าที่ยังไม่ทำคือ**หน้าที่มีฟอร์ม** (ยืม คีน ตรวจสอบภาพ อุทธรณ์) ซึ่งเป็นหน้าที่ผูกกับ zod มากที่สุด นี่คือเหตุผลที่หัวข้อ 13 ยืนยันว่าต้องยก zod **ตอนนี้** ไม่ใช่ตอนหน้าฟอร์มเสร็จหมดแล้ว

6 จุดที่ 1 — หน้าล็อกอินเดาบทบาทเอง

นี่คือจุดที่ร้ายแรงที่สุดในฝั่ง frontend และเป็นจุดที่ข้อมูลจำลองปิดบังไว้ได้สนิท

โค้ดที่อยู่ในโปรตอนนี้ · login-page.tsx:40--43

```
// KU email → borrower only (students & faculty).
const handleKuLogin = () => {
  loginAs("borrower");
  navigate(HOME_ROUTE_BY_ROLE.borrower, { replace: true });
};
```

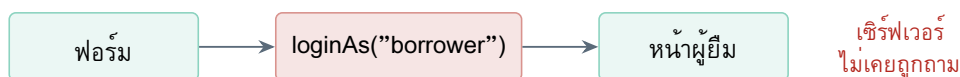
คอมเมนต์เขียนไว้เองว่า “students & faculty” แต่โค้ดได้คอมเมนต์ กำหนดบทบาทเป็น borrower ตายตัว

ปัญหาคืออาจารย์ทุกคนก็ใช้อีเมล KU เมื่อทดสอบกับฐานข้อมูลจริง บัญชี orawan.p@ku.th มี RoleName เป็น Professor ซึ่งแปลงเป็นบทบาท supervisor ไม่ใช่ borrower

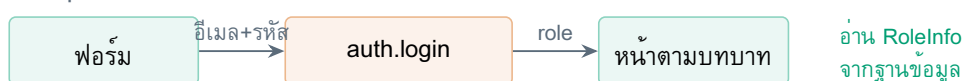
โค้ดเดิมจึงจะจับอาจารย์ทั้งคณะยัดเข้าหน้าผู้ยืม พร้อมสิทธิ์ที่ผิด และไม่มีอะไรจับได้ เพราะข้อมูลจำลองไม่เคยมีอาจารย์ที่ล็อกอินด้วยอีเมล KU

รูปที่ 3 อธิบายว่าความผิดพลาดนี้เกิดจากการวาง “ผู้ตัดสินใจ” ไว้ผิดฝั่ง

วิธีเดิม — เบราร์เซอร์ตัดสินใจเอง



วิธีที่ spike ใช้ — เซิร์ฟเวอร์ตัดสินใจ



รูปที่ 3: ความต่างไม่ได้อยู่ที่จำนวนบรรทัด แต่อยู่ที่ว่าใครเป็นคนตอบคำถาม “คนนี้เป็นใคร”

spike แก่เป็นอย่างนี้ · ทดสอบผ่านในเบราร์เซอร์จริงแล้ว

```
const submit = async (method, identifier, password) => {
  const user = await login.mutateAsync({ method, identifier, password });
  navigate(HOME_ROUTE_BY_ROLE[user.role], { replace: true });
  return null;
};
```

ปลายทาง มา จาก user.role ที่ เซิร์ฟเวอร์ ตอบ ไม่ใช่ ค่าที่ หน้า เว็บ รู้ ล่วง หน้า ผล การ ทดสอบ: orawan.p@ku.th ไปไฟล์ที่ /supervisor/approvals ถูกต้อง

7 จุดที่ 2 — บทบาทเก็บอยู่ใน localStorage

จุดนี้รุนแรงกว่าจุดแรก เพราะไม่ใช่แค่ “เดาผิด” แต่เป็น “ให้ผู้ใช้เลือกเองได้”

PO ช่องโหว่ · auth.store.ts:36, 46

```
// login
window.localStorage.setItem(MOCK_USER_STORAGE_KEY, role);
// on app start
role = window.localStorage.getItem(MOCK_USER_STORAGE_KEY);
set({ user: isRole(role) ? MOCK_USERS[role] : null, isLoading: false });
```

localStorage แก่ได้จาก DevTools ของเบราว์เซอร์ในสามวินาที ใครก็ตามที่เปิดหน้าเว็บนี้ พิมพ์คำเดียว ก็เป็น admin ได้

ตอนนี้ยังไม่อันตรายจริง เพราะหลังบ้านยังไม่มีข้อมูลจริงให้เข้าถึง แต่วันที่ backend ต่อเสร็จ ช่องนี้จะกลายเป็นประตูหลังทันที ถ้าไม่มีใครจำได้ว่าต้องเอาออก

spike แก่โดยลบทั้งสองฟังก์ชันทิ้ง แล้วให้เซิร์ฟเวอร์เป็นผู้ตอบแทน ผ่านคุกกี้ httpOnly ซึ่ง JavaScript อ่านไม่ได้เลย — รายละเอียดตกลงไถ่อยู่ในภาค 4

ประโยชน์ที่ร้อยจุดที่ 1 กับจุดที่ 2 เข้าด้วยกัน

ทั้งสองจุดคือความผิดพลาดชนิดเดียวกัน: ให้เบราว์เซอร์เป็นผู้ตัดสินใจ เรื่องที่เบราว์เซอร์ไม่มีสิทธิ์ตัดสินใจ

หลังแก้แล้ว ไม่มีอะไรในเบราว์เซอร์แจกลสิทธิ์ให้ตัวเองได้อีก นี่คือนโยบายที่จับต้องได้ที่สุดของการต่อ backend จริง มากกว่าเรื่อง “ได้ข้อมูลจริงมาแสดง”

8 จุดที่ 3 — รหัสผ่านทดสอบอยู่ในรีโป

frontend/src/features/auth/mock-auth.ts:62--70 เก็บตารางบัญชีและรหัสผ่าน เป็นข้อความธรรมดา และ frontend/README.md:14--18 พิมพ์ซ้ำอีกรอบเป็นตาราง

ทั้งสองที่มีคำเตือนเขียนกำกับไว้แล้วว่าให้ลบเมื่อ backend มา ซึ่งดีมาก แต่คำเตือนไม่ใช่กลไก — ไม่มีอะไรบังคับให้ลบจริง

spike ลบ mock-auth.ts ทั้งทั้งไฟล์ (70 บรรทัด) บัญชีทดสอบย้ายไปอยู่ใน seed.ts ของฐานข้อมูล ซึ่งเป็นที่ถูกต้อง เพราะรหัสผ่านถูก hash ด้วย scrypt ก่อนเก็บ ไม่ได้เป็นข้อความธรรมดาอีกต่อไป

9 จุดที่ 4 — ฟอรัมต้องกลายเป็น async

จุดนี้ไม่ใช่ความผิดพลาด แต่เป็นผลกระทบลูกโซ่ที่ทีมควรรู้ง่วงหน้า เพราะจะเกิดซ้ำกับทุกฟอรัมที่เหลือ ตอนที่การล็อกอินเป็นของจำลอง มันตอบทันที ฟังก์ชันจึงเขียนแบบ synchronous ได้

```
// before: the mock answers instantly
onSubmit: (values: LocalLoginValues) => string | null | void;
// after: a real round trip
onSubmit: (values: LocalLoginValues) => Promise<string | null | void>;
```

การเปลี่ยนคำเดียวนี้ลากของตามมามากมายในทุกรูปแบบ

1. ตัวเรียกต้อง `await` และตัวจัดการต้องเป็น `async`
2. ปุ่มต้องมีสถานะ “กำลังส่ง” เพราะคำขอจริงซ้ำพ้อที่ผู้ใช้จะกดซ้ำ
3. ต้องมีที่แสดงข้อความผิดพลาดจากเซิร์ฟเวอร์ ซึ่งฟอร์มเดิมไม่มี

ข้อ 3 เห็นชัดที่ช่องอีเมล KU: ตอนเป็นของจำลอง เส้นทางนั้นล้มเหลวไม่ได้เลย จึงไม่เคยมีที่ให้แสดง “รหัสผ่านไม่ถูกต้อง” พอดีของจริงกลายเป็นกรณีที่พบบ่อยที่สุด

คุณด้วยจำนวนฟอร์มที่ยังไม่ได้ทำ

งานสามข้อข้างบนใช้เวลาไม่ก่นาที่ต่อฟอร์ม แต่ต้องทำทุกฟอร์ม และหน้าที่ยังเป็น placeholder อีก 19 หน้าส่วนใหญ่มีฟอร์ม ถ้าทีมเขียนฟอร์มที่เหลือแบบ synchronous ต่อไปโดยยังไม่ต่อ backend งานรื้อรอบเดียวจะใหญ่กว่านี้หลายเท่า

10 จุดที่ 5 — แถบข้างแสดงชื่อที่ตายตัว

`sidebar.tsx` แสดง `persona.name` ซึ่งมาจาก `constants/navigation.ts` — เป็นค่าคงที่หนึ่งชุดต่อหนึ่งบทบาท ไม่ใช่ชื่อของคนที่ล็อกอินจริง

ตอนนี้ยังไม่มีใครเห็นความต่าง เพราะ `seed.ts` ตั้งชื่อให้ตรงกับข้อมูลจำลองพอดี แต่พอดูให้ละเอียดจะพบว่าไม่ตรง: ฐานข้อมูลเก็บว่า “อรรณณ ภักดี” ส่วนแถบข้างเขียนว่า “ผศ.ดร. อรรณณ ภักดี”

spike ไม่ได้แก้จุดนี้ เพราะเป็นงานของเจ้าของ AppShell ไม่ใช่ของระบบล็อกอิน จัดเป็น P2 — แก่ตอนแตะไพล์นั้นพอดี

11 สรุปสถานที่ที่ 1

ตารางที่ 3: ห้าจุดของสถานที่ที่ 1 และสถานะ

ระดับ	จุด	spike แก่แล้ว	ไฟล์
P0	บทบาทเก็บใน <code>localStorage</code>	ใช่	<code>auth.store.ts</code>
P0	รหัสผ่านทดสอบอยู่ในรีโป	ใช่ (ลบทั้งไฟล์)	<code>mock-auth.ts</code>
P1	หน้าล็อกอินเดาบทบาทเอง	ใช่	<code>login-page.tsx</code>
P2	ฟอร์มต้องเป็น <code>async</code>	ใช่ (2 ไฟล์)	<code>login-method-*.tsx</code>
P2	แถบข้างแสดงชื่อตายตัว	ยังไม่แก้	<code>sidebar.tsx</code>

ภาค 3

สถานีที่ 2 — สัญญาและ type

สถานีนี้คือรอยต่อที่แท้จริง และเป็นທີ່ที่ค่าผ่านทางทั้งหมดของทางเลือก ข. อยู่

12 สัญญาเดินทางมาแบบไหน

ในทางเลือก ข. “สัญญา” คือไฟล์หนึ่งไฟล์ที่เครื่องมือสร้างจาก router ของ backend แล้วคัดลอกมาวางไว้ใน frontend/src/generated/server.ts

ไฟล์นั้น import แค่อสองอย่าง

```
import { initTRPC } from "@trpc/server";  
import { z } from "zod";
```

บรรทัดที่สองคือต้นเหตุของทุกอย่างในภาคนี้

13 จุดที่ 6 — zod คนละรุ่น

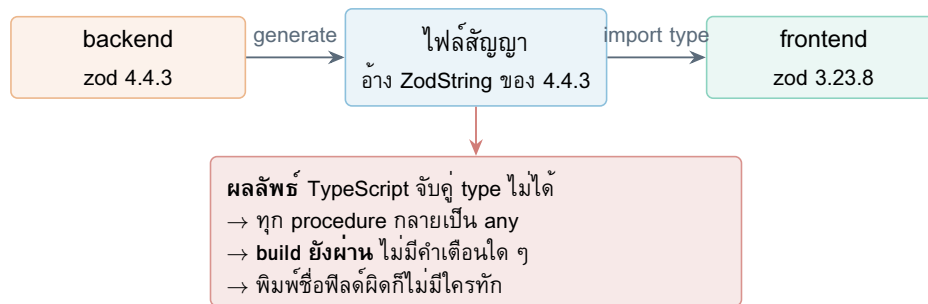
P1 ตัวบล็อกที่เจอตังแต่ยังไม่เขียนโค้ดบรรทัดแรก

frontend/package.json:40 ประกาศ "zod": "^3.23.8"
spike-option-b/backend/package.json ประกาศ "zod": "4.4.3"
ไฟล์สัญญาที่ generate ออกมาผูกกับรุ่นของฝั่งที่สร้างมัน คือ 4.4.3

ทำไมรุ่นต่างกันแล้วพัง — และทำไมมันพังเงียบ

TypeScript ตัดสินว่า type สองตัวเหมือนกันหรือไม่จากโครงสร้าง แต่ ZodString ของ zod@3 กับของ zod@4 มีโครงสร้างภายในต่างกัน ทั้งคู่ชื่อ ZodString เหมือนกันแต่ไม่ใช่ตัวเดียวกัน ผลคือการอ่าน type ออกจากสัญญาจะล้มเหลว แล้วทุก procedure ตกเป็น any

any ไม่ทำให้ build พัง มันแค่แปลว่า “ไม่ต้องตรวจอะไรเลย” ซึ่งคือการสูญเสียเหตุผลทั้งหมดที่เลือกใช้ tRPC ตั้งแต่แรก โดยที่ไม่มีข้อความเตือนสักบรรทัด



รูปที่ 4: เหตุผลที่ zod คนละรุ่นเป็นความล้มเหลวแบบเงียบ ไม่ใช่ข้อผิดพลาดที่มองเห็น

13.1 ขาวดี: ยกตอนนี้อย่างไร

ทดลองแล้ว ยก zod เป็น 4.4.3 โดยยังไม่แตะโค้ดสักบรรทัด

ผลการรัน `tsc --noEmit` คือ ผ่าน 0 ข้อผิดพลาด
 เหตุผล: ทั้งโปรเจกต์ใช้ zod แค่ 2 ไฟล์ คือ `features/auth/login.schema.ts` กับ `schemas/loan-request.schema.ts` และทั้งสองใช้แค่ API ที่ zod@4 ยังรองรับ (`.email()` และ `.datetime()` ถูกประกาศเลิกใช้แล้วแต่ยังทำงานได้)

การยก zod ลากของตามมาหนึ่งอย่าง คือ `@hookform/resolvers` ต้องขึ้นจาก `^3.10.0` เป็น `5.2.2` เพราะรุ่น 3 รองรับเฉพาะ zod@3

ขาวดีนี้มีวันหมดอายุ

ที่ยกได้ฟรีวันนี้เพราะมี schema แค่ 2 ไฟล์
 หน้าที่ยังไม่ทำอีก 19 หน้าคือหน้าที่มีฟอร์ม แต่ละฟอร์มจะเพิ่ม schema ต้นทุนการยกจึงโตทุกสัปดาห์ที่เลื่อนออกไป
 ถ้าทีมจะเลือกทางนี้ ต้องยก zod ตอนนี้อย่างไรตอนหน้าฟอร์มเสร็จหมดแล้ว

14 จุดที่ 7 — type ของผู้ใช้ที่เขียนด้วยมือ

`frontend/src/types/domain.ts:9--18` เขียน `interface User` ขึ้นเอง โดยอ้างอิงความเข้าใจ ณ ตอนนั้น พอเทียบกับสิ่งที่เซิร์ฟเวอร์ส่งจริง พบว่าผิด 3 ฟิลด์

ข้อ `departmentId` อันตรายแบบเงียบที่สุด: โค้ดที่เอาไปต่อสตริงจะได้คำว่า `"null"` โผล่บนหน้าจอ ไม่ใช่ค่าว่าง

14.1 วิธีที่ spike ใช้กันไม่ให้เกิดอีก

เลิกเขียน `User` ด้วยมือ เปลี่ยนเป็นดึงออกมาจากสัญญาโดยตรง

```
// src/types/domain.ts
export type User = ContractUser; // from inferRouterOutputs
export type Role = ContractUser["role"];
```

ตารางที่ 4: สามฟิลด์ที่ได้อัด และเหตุผลจากฐานข้อมูลจริง

ฟิลด์	ทีมเขียนไว้	ของจริง	เพราะอะไร
id	string	number	AccountInfo.AccountKey เป็น Int @id
departmentId	string	string null	ผู้ใช้ที่ยังไม่ถูกผูกกับภาควิชาใดจะไม่มีค่า
maxBorrowDays	ไม่มีเลย	number	เซิร์ฟเวอร์คำนวณจาก CreditTier ส่งมาให้

```
export type CreditBand = ContractUser["creditBand"];
```

ผลทันทีเมื่อทำแบบนี้: tsc ชี้จุดที่ขัดกับเซิร์ฟเวอร์ให้เอง และมันชี้จริง — mock-auth.ts พัง 4 บรรทัด ซึ่งเป็นไฟล์ที่ต้องลบอยู่แล้ว

ผลที่สำคัญกว่านั้น

นอกจากไฟล์ที่ต้องลบแล้ว ไม่มีอะไรพังเลยทั้งโปรเจกต์

แปลว่ารูปร่างข้อมูลที่เซิร์ฟเวอร์ส่งมาสมกับโค้ดที่ทีมเขียนไว้ได้พอดี งานต่อ backend จึงไม่ใช้การรื้อ แต่เป็นการเสียบ

15 จุดที่ 8 — รหัสข้อผิดพลาดคนละชุด

frontend/src/lib/error-messages.ts:11 เตรียมข้อความไว้สำหรับรหัส INVALID_DOMAIN

backend/src/contract.types.ts ส่งรหัส NOT_KU_EMAIL

คนละรหัส ทั้งสองฝั่ง build ผ่าน ไม่มีใครเตือน

ผู้ใช้ที่กรอกอีเมล gmail จะเห็นข้อความ “เกิดข้อผิดพลาดที่ไม่ทราบสาเหตุ” แทนข้อความที่เขียนเตรียมไว้อย่างดี

สาเหตุเชิงโครงสร้าง: ตารางแปลรหัสประกาศเป็น Record<string, string> ซึ่งรับ key อะไรก็ได้ การค้นหาที่ไม่เจอไม่ใช่ข้อผิดพลาด มันแค่คืนค่าว่าง

นี่คือความต่างที่ชัดที่สุดระหว่างสองทางเลือก: ในทางเลือก ก. ฝั่ง frontend import { BUSINESS_ERROR } จากแพ็คเกจกลาง การเพิ่มรหัสใหม่แล้วไม่แปล จะกลายเป็น compile error ทันที ในทางเลือก ข. ไม่มีอะไรเชื่อมสองฝั่งเลย

16 จุดที่ 9 — สัญญาเก่าค้างได้ และ CI มองไม่เห็น

ทางเลือก ข. มีความเสี่ยงเฉพาะตัวคือ ไฟล์สัญญาเป็นสำเนา ถ้าแก้ router แล้วลืมสั่ง generate ฝั่ง frontend จะยัง build ผ่าน เพราะ type เก่าก็ยังเป็น type ที่ถูกต้อง

spike จึงเขียนด้าน CI ไว้สองตัวเพื่อจับเรื่องนี้ และทดสอบแล้วว่าจับได้จริง แต่ตอนเอา frontend จริงมาต่อพบว่าด้านทั้งสองมองไม่เห็นมัน

P2 ทดลองทำให้พังแล้ว ด้านยังเขียวทั้งคู่

ตั้ง frontend-real ให้ประกาศ zod เป็น ^4.7.0 (ผิดทั้งรุ่นและไม่ pin) และแก้ไฟล์สัญญาให้เพี้ยนจากต้นฉบับ แล้วรันด้านทั้งสอง

```
check-versions: PASS
check-contract-fresh: PASS
```

สาเหตุ: สคริปต์ทั้งคู่ระบุ path ไว้ตรง ๆ ว่า frontend/ ผู้บริโภคสัญญารายที่สองจึงอยู่นอกสายตา

นี่ไม่ใช่บั๊กของสคริปต์ แต่เป็นรูปร่างของปัญหา

ทางเลือก ข. ต้องมีด้านหนึ่งด้านต่อผู้บริโภคหนึ่งราย และต้องมีคนจำได้ว่าต้องเพิ่มด้านเมื่อมีผู้บริโภครายใหม่ (แอปมือถือ, admin portal, ชุดทดสอบ e2e)

ทางเลือก ก. ไม่มีคำถามนี้ เพราะ npm workspaces ยุบ zod ให้เหลือก่อนเดียวโดยโครงสร้าง ไม่ใช่โดยวินัย

ถ้าเพิ่มเลือกทางนี้ ต้องแก้สคริปต์ให้วนทุกโฟลเดอร์ที่กินสัญญา ไม่ใช่เขียนชื่อเดียวตายตัว

17 สรุปสถานที่ที่ 2

ตารางที่ 5: หัวจุดของสถานที่ที่ 2 และสถานะ

ระดับ	จุด	spike แก่แล้ว	ไฟล์
P1	zod 3 ต้องยกเป็น 4 (ลาก @hookform/resolvers 3→5 ตามมาด้วย)	ใช่	package.json
P1	User เขียนมือ ผิด 3 ฟิลด์	ใช่	types/domain.ts
P2	รหัสข้อผิดพลาดคนละชุด	ใช่	error-messages.ts
P2	CI มองไม่เห็นผู้บริโภครายที่สอง	ยังไม่แก้	scripts/*.mjs

ภาค 4

สถานที่ที่ 3 — เครือข่ายและ session

สถานที่นี้คือระยะทางระหว่างเบราว์เซอร์กับเซิร์ฟเวอร์ สิ่งที่เกิดขึ้นบนเส้นทางนี้ ไม่ได้มีแค่ข้อมูล แต่มีตัวตนของผู้ใช้ด้วย

18 ทีมเตรียมอะไรไว้แล้ว

ข้อดี: frontend/src/lib/api-client.ts:42 ตั้ง credentials: "include" ไว้แล้วพร้อมคอมเมนต์ว่า “ส่ง cookie ทุกครั้ง” แปลว่าทีมตัดสินใจเรื่อง รูปแบบการยืนยันตัวตนถูกต้องมาตั้งแต่ต้น — ใช้คุกกี้ ไม่ใช่ token ใน localStorage

frontend/README.md ก็เขียนไว้ชัดว่า “Auth: httpOnly cookie (session-based)”

การตัดสินใจที่ถูกแล้ว ไม่ต้องเปลี่ยน

เลือกคุกกี้ httpOnly แทนการเก็บ token ใน localStorage เป็นการตัดสินใจที่ถูกและสำคัญ เพราะ JavaScript อ่านคุกกี้แบบนี้ไม่ได้เลย สคริปต์ที่ถูกฝังในหน้าเว็บจึงขโมยไปไม่ได้ spike ทดสอบยืนยันในเบราว์เซอร์จริงแล้วว่า document.cookie มองไม่เห็น ulms_session จริง ๆ

19 กลไกที่เกิดขึ้นจริงเมื่อล็อกอิน

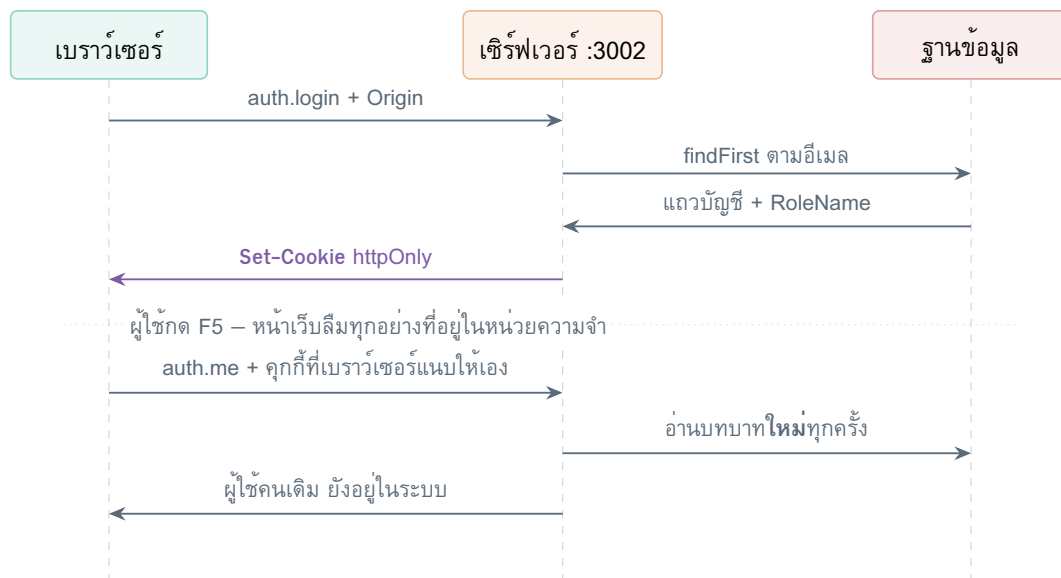
รูปที่ 5 คือลำดับที่เกิดขึ้นจริง วัดจากการทดสอบ ไม่ใช่ภาพอุดมคติ

จุดที่ควรสังเกตในรูปคือลูกศรเส้นที่ห้า: หลังรีเฟรช เบราวเซอร์แนบคุกกี้ให้เอง โดยที่โค้ดไม่ต้องทำอะไรเลย และเซิร์ฟเวอร์อ่านบทบาทใหม่จากฐานข้อมูลทุกครั้ง ไม่ได้ฝังบทบาทไว้ในคุกกี้

การอ่านใหม่ทุกครั้งช้ากว่า แต่แลกมาด้วยความถูกต้อง: วันที่ผู้ดูแลอนสิทธิ์ใคร คนนั้นเสียสิทธิ์ทันที ไม่ใช่ตอนคุกกี้หมดอายุ

20 จุดที่ 10 — CORS ที่ยังไม่มี

backend/src/main.ts ของทีมตอนนี้มี 7 บรรทัด และไม่มี CORS ไม่มี cookie-parser



รูปที่ 5: ลำดับจริงของการล็อกอินและการกู้ session คีนหลังรีเฟรช — ทั้งสองเส้นทางถูกทดสอบในเบราว์เซอร์จริงแล้ว

P1 ถ้าไม่ตั้งสองอย่างนี้ ทุกอย่างก็เหลือใช้ไม่ได้

```
// backend/src/main.ts as it stands today
const app = await NestFactory.create(AppModule);
await app.listen(process.env.PORT ?? 3000);
```

เบราว์เซอร์จะทิ้งคำตอบทุกอันก่อนที่จะได้ฝั่งหน้าเว็บจะได้เห็น และเซิร์ฟเวอร์จะอ่านคุกกี้ที่ส่งมาไม่ได้

สิ่งที่ต้องเพิ่ม และเหตุผลของแต่ละบรรทัด

```
app.use(cookieParser()); // without this, req.cookies is undefined
app.enableCors({
  origin: allowedOrigins, // must be an explicit list, NOT "*"
  credentials: true, // without this the browser drops the cookie
  methods: ['GET', 'POST', 'OPTIONS'],
});
```

กับดักที่คนเจอบ่อยที่สุดเรื่อง CORS กับคุกกี้

เมื่อ `credentials: true` มาตรฐานของเบราว์เซอร์ห้ามใช้ `origin: "*"` ต้องระบุที่อยู่ให้ครบทุกอันแบบเจาะจง

แปลว่าทุกครั้งที่มีคนรัน frontend บนพอร์ตใหม่ ต้องไปเพิ่มชื่อในฝั่งเซิร์ฟเวอร์ด้วย — ในการทดลองครั้งนี้ต้องเพิ่ม `http://localhost:5275` เข้าไปจริง ๆ ก่อนถึงจะล็อกอินได้

21 จุดที่ 11 — proxy ที่ซัฟฟิดที่

frontend/vite.config.ts ตั้ง proxy ให้ `/api` วิ่งไปที่ `localhost:3000` ซึ่งเป็นการตั้งค่าสำหรับ REST

ไม่ใช่ tRPC

ถ้าใช้ tRPC แบบที่ spike ทำ คือคุยตรงกับ :3002 พร้อม CORS proxy ตัวนี้จะไม่ถูกใช้เลย จัดเป็น P2 — ไม่ได้ฟัง แต่ทำให้สับสน และควรตัดสินใจให้ตรงกันว่าจะใช้แบบไหน

22 จุดที่ 12 — แยก “ต่อไม่ติด” ออกจาก “รหัสผิด” ไม่ได้

error-messages.ts เดิมไม่มีทางแยกสองกรณีนี้ ผลคือ ถ้าเซิร์ฟเวอร์ดับ ผู้ใช้จะเห็นข้อความว่ารหัสผ่านไม่ถูกต้อง ซึ่งเป็นการโกหก และทำให้ผู้ใช้พิมพ์รหัสซ้ำอีกสักรอบโดยไม่มีทางสำเร็จ

spike แก้โดยดูว่าคำตอบมีข้อมูลจากเซิร์ฟเวอร์แนบมาหรือไม่

```
export function isConnectionError(error: unknown): boolean {
  // tRPC still throws TRPCClientError on transport failure, but with no
  // server payload attached. That absence is the signal.
  return error instanceof TRPCClientError && !error.data;
}
```

ทดลองได้เองใน 30 วินาที

ปิด backend แล้วลองล็อกอิน ต้องขึ้นข้อความว่า “เชื่อมต่อเซิร์ฟเวอร์ไม่ได้ กรุณาตรวจสอบว่า backend ทำงานอยู่” ไม่ใช่ “ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง”

23 สรุปสถานที่ 3

ตารางที่ 6: สามจุดของสถานที่ 3 และสถานะ

ระดับ	จุด	spike แก่แล้ว	ไฟล์
P1	main.ts ไม่มี CORS / cookie-parser	ใช่ (ในฝั่ง spike)	backend/src/main.ts
P2	แยกกรณีต่อไม่ติดไม่ได้	ใช่	lib/trpc.ts
P2	proxy /api ชี้ :3000	ยังไม่ตัดสินใจ	vite.config.ts

ภาค 5

สถานีที่ 4 — server

24 ทีมมีอะไรตอนนี้

คำตอบสั้น ๆ คือ ยังไม่มีระบบล็อกอินเลย ไม่ใช่ “มีแต่ยังไม่เสร็จ” แต่คือยังไม่ได้เริ่ม

backend/src/ ที่ไม่ใช่ไฟล์ที่เครื่องมือสร้างให้ มีอยู่ 9 ไฟล์ และเป็นโครงเปล่าของ NestJS กับการต่อ Prisma ทั้งหมด

ตารางที่ 7: ไฟล์ทั้งหมดใน backend/src/ ที่คนเขียนเอง

ไฟล์	คืออะไร
main.ts	7 บรรทัด ไม่มี CORS ไม่มี cookie-parser
app.module.ts	โครงเปล่า ยังไม่ได้ตั้ง TRPCModule
app.controller.ts	controller ตัวอย่างที่ Nest สร้างให้ตอน new
app.service.ts	service ตัวอย่าง
prisma.service.ts	ต่อฐานข้อมูล — ส่วนนี้ใช้ได้จริง
prisma.module.ts	module ของข้างบน
querytest.ts	สคริปต์ลองคิวรี
*.spec.ts	ไฟล์ทดสอบตัวอย่าง 2 ไฟล์

25 จุดที่ 13 — แดไฟล์ที่ต้องเขียนเพิ่ม

spike เขียนไฟล์เหล่านี้ขึ้นมาแล้วและใช้งานได้จริง ยกมาได้เลย รายการนี้คือ “ระยะห่าง” ผัง backend ที่เป็นรูปธรรมที่สุด

ไฟล์ที่สำคัญที่สุดในตารางคือ user.mapper.ts

เป็นที่เดียวในระบบที่รู้จักทั้ง “รูปร่างของ Prisma” และ “รูปร่างของสัญญา” ทุก procedure ที่ส่งข้อมูลผู้ใช้ออกไปต้องผ่านที่นี่

เหตุผลไม่ใช่ความเป็นระเบียบ แต่เป็นความปลอดภัย: ตาราง AccountInfo เก็บ HashedPassword ไว้ในแถวเดียวกัน การคืนแถว Prisma ออกไปตรง ๆ จะทำให้รหัสผ่านที่ hash แล้วหลุดไปถึงเบราว์เซอร์

ตารางที่ 8: ไฟล์ที่ backend ต้องมีเพิ่มเพื่อให้ล็อกอินได้ อ้างจาก spike-option-b/backend/src/

ไฟล์	หน้าที่ และเหตุผลที่ต้องแยกไฟล์
auth/auth.router.ts	นิยาม 3 procedure พร้อม schema — เป็นไฟล์เดียวที่เครื่องมืออ่านเพื่อสร้างสัญญา
auth/auth.service.ts	ตรรกะการล็อกอิน ไม่รู้จัก trRPC คูกี้ หรือ HTTP เลย จึงทดสอบได้โดยไม่ต้องยิงคำขอ
auth/password.ts	hash และตรวจรหัสผ่านด้วย scrypt
auth/session.ts	ออกและตรวจ token + ตั้งค่าคูกี้
auth/auth.module.ts	ประกอบสามไฟล์ข้างบนเข้า Nest
trpc/context.ts	แปลงคูกี้ในคำขอเป็น “ผู้ใช้คนไหนคือใคร”
trpc/auth.middleware.ts	ด้านที่ปฏิเสธคำขอที่ยังไม่ล็อกอิน
common/user.mapper.ts	แปลงแถว Prisma เป็นรูปร่างตามสัญญา

26 บทเรียนจากโค้ดจริง — การกันการเดาบัญชี

หัวข้อนี้ไม่ใช่จุดที่ต้องแก้ แต่เป็นสิ่งที่ทีมควรรู้จักก่อนเขียน auth.service.ts เอง เพราะเป็นความผิดพลาดที่เขียนเองแล้วพลาดกันบ่อยมาก

ถ้าเขียนตรงไปตรงมา โค้ดจะคืนค่าทันทีเมื่อหาบัญชีไม่เจอ

```
// the obvious version - and it leaks information
const account = await findAccount(identifier);
if (!account) throw new TRPCError({ code: 'UNAUTHORIZED' });
const ok = await verifyPassword(input.password, account.HashedPassword);
```

ปัญหาคือกรณี “ไม่มีบัญชีนี้” จะตอบเร็วกว่ากรณี “รหัสผิด” อย่างเห็นได้ชัด เพราะไม่ได้รัน scrypt ซึ่งจึงใจให้ช้า คนที่ยิงอีเมลไปเรื่อย ๆ แล้วจับเวลาจะรู้ได้ว่าอีเมลไหนมีบัญชีอยู่จริง

วิธีที่ spike ใช้คือรัน scrypt กับค่าหลอกเสมอ ให้เวลาเท่ากันทั้งสองกรณี

```
const stored = account?.HashedPassword ?? DUMMY_HASH;
const ok = await verifyPassword(input.password, stored);
if (!account || !ok) throw new TRPCError({ code: 'UNAUTHORIZED',
message: 'INVALID_CREDENTIALS' });
```

สังเกตว่าข้อความที่ตอบกลับเหมือนกันทั้งสองกรณีโดยตั้งใจ — ห้ามบอกผู้เข้าว่า “ไม่มีบัญชีนี้”

27 สรุปสถานที่ 4

ตารางที่ 9: จุดเดียวของสถานที่ 4 — แต่เป็นจุดที่ใหญ่ที่สุดในเอกสารเมื่อวัดเป็นปริมาณงาน

ระดับ	จุด	spike แก่แล้ว	ที่ยกมาใช้ได้
P0	backend ยังไม่มีระบบล็อกอินเลย ต้องเขียนเพิ่ม 8 ไฟล์	ใช่ (เขียนครบและรันได้)	spike-option-b/backend/src/

ภาค 6

สถานีที่ 5 — ฐานข้อมูล

นี่คือภาคที่สำคัญที่สุดของเอกสาร ทุกจุดในภาคนี้มีลักษณะร่วมกันสองอย่าง: **spike แก้ไม่ได้** เพราะต้องเขียน migration ซึ่งกระทบทุกคน และ**ยิ่งเลื้อนยิ่งแพง** เพราะทุกตารางที่ถูกใช้งานเพิ่มทำให้การแก้ยากขึ้น

28 สภาพปัจจุบัน

backend/prisma/schema.prisma มี 33 model และ migration เพียงชุดเดียวคือ 20260805125233_init spike ใช้สำเนาของไฟล์นี้ ไม่ได้ชี้ไปที่ไฟล์จริง และตรวจแล้วว่า เนื้อหาตรงกันทุกบรรทัด ต่างกันแค่คอมเมนต์ที่เติมไว้ตอนต้นไฟล์ ข้อค้นพบทั้งหมดในภาคนี้จึงใช้กับฐานข้อมูลจริงของทีมได้โดยตรง

29 PO จุดที่ 14 — คอลัมน์ที่ใช้ล็อกอินไม่มี @unique

ช่องโหว่ที่ร้ายแรงที่สุดในเอกสารนี้ · schema.prisma:22, 24

```
model AccountInfo {
  AccountKey Int @id @default(autoincrement())
  Email String // <-- no @unique
  HashedPassword String
  UserID String // <-- no @unique
  ...
}
```

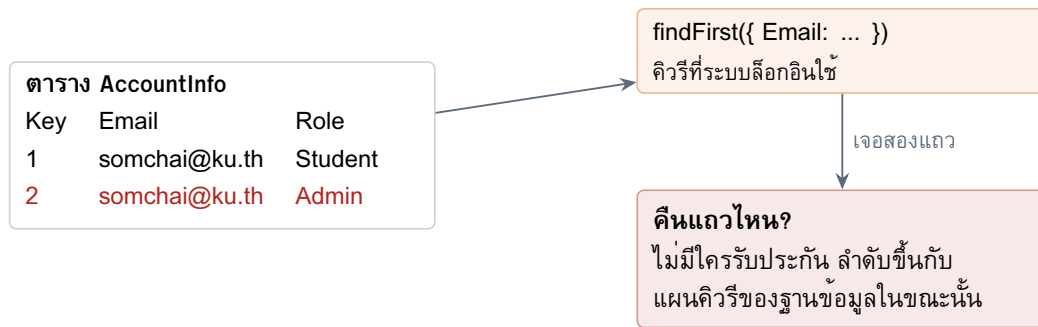
ฐานข้อมูลอนุญาตให้มีอีเมลซ้ำกันได้ และอนุญาตให้มี UserID ซ้ำกันได้ ทั้งสองคอลัมน์คือคอลัมน์ที่ระบบล็อกอินใช้ค้นหาผู้ใช้

ผลที่ตามมาในโค้ดเห็นได้ทันที: auth.service.ts ต้องใช้ findFirst ไม่ใช่ findUnique เพราะ Prisma ไม่ยอมให้ค้นแบบ unique กับคอลัมน์ที่ไม่ได้ประกาศว่า unique และ “first” แปลว่าแถวไหนก็ได้ที่เจอก่อน

migration ที่ต้องเขียน ข้อที่ 1

เติม @unique ให้ Email และ UserID

ต้องทำก่อนมีข้อมูลจริง เพราะถ้ามีแถวอยู่แล้ว migration จะรันไม่ผ่าน และต้องมาไล่ลบข้อมูลทีหลัง ตอนนี้ฐานข้อมูลยังว่าง — นี่คือช่วงเวลาที่ถูกที่สุด



สถานการณ์โจมตี สมัครงับด้วยอีเมลของคนอื่น แล้วรอให้ระบบเลือกแถวของเรา — หรือแย่กว่านั้น ให้ระบบเลือกแถวของ Admin ตอนเจ้าตัวล็อกอิน

รูปที่ 6: ผลของการไม่มี @unique บนคอลัมน์ที่ใช้ล็อกอิน — ปัญหานี้ไม่ได้อยู่ที่โค้ด แต่อยู่ที่โครงสร้างตาราง

30 PO จุดที่ 15 — ไม่มีตารางเก็บ session

ค้นทั้ง 33 model ไม่พบตารางชื่อ Session, Token, RefreshToken หรืออะไรที่ทำหน้าที่นี้เลยแม้แต่ตารางเดียว

นั่นทำให้เหลือทางเลือกสองทาง และทั้งสองทางมีราคา

ตารางที่ 10: สองทางเลือกเรื่อง session และสิ่งที่ต้องแลก

	ก. JWT ไร้สถานะ	ข. เพิ่มตาราง Session
spike ใช้	ใช่	ไม่
ต้อง migrate	ไม่ต้อง	ต้อง — แต่งานคนอื่น
ทำได้เมื่อไร	วันนี้	ต้องตกลงกันก่อน
เพิกถอน session	ทำไม่ได้จริง — “ออกจากระบบ” แคลบคูกก็ในเบราว์เซอร์ token นั้น ยังใช้ได้จนหมดอายุ	ทำได้ และดูได้ว่าผู้ใช้ล็อกอินจากที่ไหนบ้าง
ต้นทุนต่อคำขอ	ไม่มีคิวรีเพิ่ม	คิวรีเพิ่ม 1 ครั้งทุกคำขอ

ประโยชน์ที่ต้องเข้าที่ประชุม

ถ้าเลือกทาง ก. ต้องยอมรับอย่างเป็นทางการว่า ถ้ามีคนขโมย token ไปได้ เราหยุดมันไม่ได้ นอกจากเปลี่ยน JWT_SECRET ซึ่งจะเตะผู้ใช้ทุกคนออกจากระบบพร้อมกัน
ระบบนี้ให้ยืมครุภัณฑ์ที่มีมูลค่าและมีขั้นตอนอนุมัติ การยอมรับข้อนี้ ต้องเป็นการตัดสินใจร่วม ไม่ใช่ผลพลอยได้จากการที่ไม่มีใครสังเกตว่าไม่มีตาราง

31 P1 จุดที่ 16 — จำนวนวันที่ยืมได้ ไม่ได้ขึ้นกับเครดิตอย่างเดียว

frontend/src/constants/index.ts:44- - 49 เขียนตารางนี้ไว้

```
export const CREDIT_BANDS = [
```

```
{ band: "D0", min: 80, max: 100, loanDays: 14, label: "..."},
{ band: "D1", min: 50, max: 79, loanDays: 7, label: "..."},
{ band: "D2", min: 30, max: 49, loanDays: 7, label: "..."},
{ band: "D3", min: 0, max: 29, loanDays: 5, label: "..."},
};
```

แต่ฐานข้อมูลบอกอีกอย่าง

```
model BorrowConstraints {
  BorrowRuleKey Int
  CreditTierKey Int
  MaxBorrowDate Int
  @@unique([BorrowRuleKey, CreditTierKey]) // <-- a PAIR, not one key
}
```

MaxBorrowDate ผูกกับคู่ (กฎการยืมของสิ่งของ × ระดับเครดิต) ไม่ใช่ระดับเครดิตอย่างเดียว

คำถามที่ไม่มีคำตอบเดียว

“ผู้ใช้คนนี้ยืมได้กี่วัน” ถ้าไม่บอกว่าจะยืมอะไร เป็นคำถามที่ตอบไม่ได้

ตาราง CREDIT_BANDS จึงผิดมาตั้งแต่บรรทัดแรกที่เขียน และผิดแบบเงียบ เพราะข้อมูลจำลองก็ตอบตามตารางนั้นอยู่แล้ว

spike แก้ปัญหาเฉพาะหน้าโดยให้เซิร์ฟเวอร์ส่ง maxBorrowDays ที่เป็นค่าสูงสุดที่เป็นไปได้ มาให้แสดงคร่าว ๆ พร้อมคอมเมนต์กำกับว่า ตัวเลขที่ผูกพันจริงต้องมาจากตอนสร้างคำขอยืม เมื่อรู้แล้วจะยืมขึ้นไหน

ทดสอบแล้วว่าตัวเลขมาจากฐานข้อมูลจริง

บัญชี test_lowcredit เครดิต 20 ได้ creditBand: "D3" และ maxBorrowDays: 5 ซึ่งอ่านมาจากตาราง ไม่ใช่จากค่าคงที่ในโค้ดหน้าเว็บ

32 P1 จุดที่ 17 — “สังกัด” ของผู้ใช้เป็นการเดา

AccountInfo ไม่มี FacultyKey เส้นทางไปหาภาควิชาต้องเดินผ่านสี่ตาราง

AccountInfo → Authority[] → ManagementGroup → FacultyInfo

และ Authority มี @@unique([AccountKey, ManageGroupKey]) ซึ่งแปลว่าคนหนึ่งคนอยู่ได้หลายกลุ่ม เช่นอาจารย์ที่ดูแลสองภาควิชา

spike เลือกเอากลุ่มแรกที่เจอ ซึ่งเป็นการเดาและเขียนกำกับไว้ตรง ๆ ในโค้ด ว่าเป็นการเดา ที่ประชุมต้องนิยามว่า “สังกัดหลัก” คืออะไร หรือเปลี่ยนให้สัญญาส่งกลับเป็นรายการ

33 P2 จุดที่ 18 — ชื่อระดับเครดิตเป็นค่าที่ไม่บังคับ

CreditTier.CreditTierName ประกาศเป็น String? คือมีหรือไม่มีก็ได้

เมื่อไม่มีชื่อ โค้ดต้องเดาระดับจากคะแนนแทน ซึ่งทำให้กฎการแบ่งระดับ ไปอยู่สองที่: ในฐานข้อมูล และในโค้ดที่เดาแทนเมื่อฐานข้อมูลไม่บอก

ทางแก้คือทำให้ฟิลด์นี้บังคับ แล้วลบตรรกะการเดาออก

34 P2 จุดที่ 19 — RoleInfo เป็นตาราง ไม่ใช่ enum

```
model RoleInfo {
  RoleKey Int @id @default(autoincrement())
  RoleName String
}
```

schema ทั้งไฟล์มี 0 enum แปลว่าชื่อบทบาทเป็นข้อความอิสระในฐานข้อมูล โค้ดต้องมีตัวแปลงจากข้อความเป็นบทบาทตามสัญญา และต้องตัดสินใจว่า ถ้าเจอชื่อที่ไม่รู้จักจะทำอย่างไร

spike เลือกให้โยนข้อผิดพลาดทันทีแทนที่จะถอยไปใช้ค่าเริ่มต้น เพราะการถอยไปใช้ค่าเริ่มต้นแปลว่าให้สิทธิ์ผิดโดยไม่มีใครรู้

```
default:
  throw new Error(`Unknown RoleName "${roleName}" - add a mapping`);
```

35 สรุปสถานที่ 5

ตารางที่ 11: หกจุดของสถานที่ 5 — ไม่มีข้อไหนที่ spike แก้ได้ เพราะทุกข้อต้องตกลงกันก่อน

ระดับ	จุด	ต้องทำอะไร
P0	Email/UserID ไม่มี @unique	migration
P0	ไม่มีตารางเก็บ session	ตัดสินใจ แล้วอาจ migrate
P1	maxBorrowDays ขึ้นกับคู่ ไม่ใช่เครดิตเดียว	แก้สัญญา + ลบ CREDIT_BANDS
P1	“สังกัดหลัก” ยังไม่ถูกนิยาม	ตัดสินใจ
P2	CreditTierName ไม่บังคับ	migration เล็ก
P2	RoleInfo เป็นตาราง	เขียนตัวแปลง + ตกลงรายชื่อบทบาท

ภาค 7

ต้องทำอะไรบ้าง และเรียงลำดับอย่างไร

36 ตารางรวมทั้ง 19 จุด

ตารางที่ 12: ทุกจุดที่พบ เรียงตามความรุนแรง — คอลัมน์สุดท้ายคือคำถามที่สำคัญที่สุด

ระดับ	จุด	สถานะ	spike แก้ให้แล้วหรือยัง
P0	Email/UserID ไม่มี @unique	ฐานข้อมูล	ไม่ — ต้องตกลงกัน
P0	ไม่มีตารางเก็บ session	ฐานข้อมูล	ไม่ — ต้องตกลงกัน
P0	บทบาทเก็บใน localStorage	หน้าจอ	ใช่
P0	รหัสผ่านทดสอบอยู่ในรีโป	หน้าจอ	ใช่
P0	backend ไม่มีระบบล็อกอิน (8 ไฟล์)	server	ใช่ ยกไปใช้ได้
P1	zod 3 ต้องยกเป็น 4	สัญญา	ใช่
P1	User เขียนมือ ผิด 3 ฟิลด์	สัญญา	ใช่
P1	หน้าล็อกอินเดาบทบาทเอง	หน้าจอ	ใช่
P1	main.ts ไม่มี CORS/cookie-parser	เครือข่าย	ใช่ ยกไปใช้ได้
P1	maxBorrowDays ขึ้นกับคู่	ฐานข้อมูล	บรรเทาชั่วคราว
P1	“สังกัดหลัก” ยังไม่ถูกนิยาม	ฐานข้อมูล	ไม่ — ต้องตกลงกัน
P2	รหัสข้อผิดพลาดคนละชุด	สัญญา	ใช่
P2	CI มองไม่เห็นผู้บริโภครายที่สอง	สัญญา	ไม่
P2	ฟอร์มต้องเป็น async	หน้าจอ	ใช่ (2 ไฟล์)
P2	แถบข้างแสดงชื่อตายตัว	หน้าจอ	ไม่
P2	แยกกรณีต่อไม่ติดไม่ได้	เครือข่าย	ใช่
P2	proxy /api ชี้ :3000	เครือข่าย	ไม่ — ต้องตัดสินใจ
P2	CreditTierName ไม่บังคับ	ฐานข้อมูล	ไม่
P2	RoleInfo เป็นตาราง	ฐานข้อมูล	บรรเทาแล้ว (มีตัวแปลง)

อ่านตารางนี้แล้วจะเห็นรูปแบบหนึ่ง

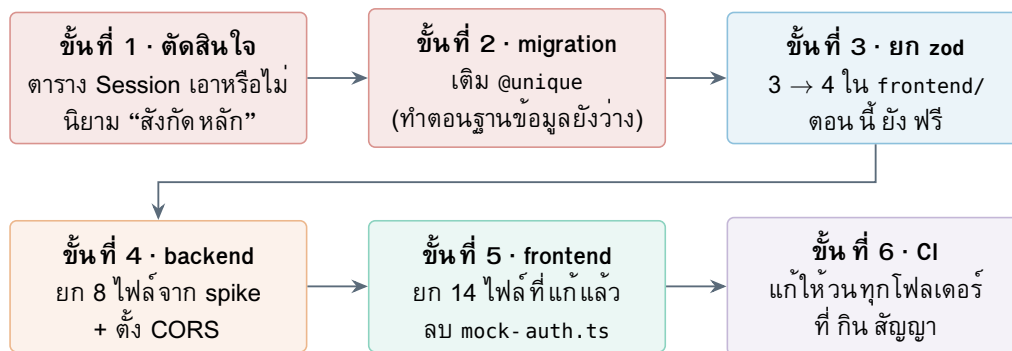
ทุกจุดที่ spike แก้ให้แล้ว คือจุดที่แก้ได้ด้วยคนคนเดียวในไฟล์ของตัวเอง

ทุกจุดที่ spike แก้ไม่ได้ คือจุดที่ต้องเตะของกลาง — schema ฐานข้อมูล หรือข้อตกลงของทีม

นั่นแปลว่างานที่เหลือส่วนใหญ่ไม่ใช่งานเขียนโค้ด แต่เป็นงานตัดสินใจ และการตัดสินใจเลื่อนได้เรื่อย ๆ จนกว่าจะมีคนกำหนดวัน

37 ลำดับที่ต้องลงมือ

ลำดับนี้ไม่ได้เรียงตามความสำคัญอย่างเดียว แต่เรียงตามการพึ่งพากัน งานบางอย่างทำก่อนแล้วถูกกว่ามาก



รูปที่ 7: ลำดับการลงมือที่เรียงตามการพึ่งพากัน ไม่ใช่ตามความสำคัญเพียงอย่างเดียว

สองประโยคที่อธิบายรูปที่ 7 ได้ครบ

- ขั้นที่ 1–2 คืองานที่ต้องทำก่อนใครเขียนโค้ดเพิ่ม เพราะเป็นการแก้ของกลาง ยังมีคนสร้างงานทับข้างบนมากเท่าไร การรื้อยิ่งแพง
- ขั้นที่ 3 ทำตอนนี้ฟรี ทำทีหลังจ่ายตามจำนวนฟอร์มที่เพิ่มเข้ามาระหว่างนั้น

ขั้นที่ 2 มีหน้าต่างเวลาที่กำลังจะปิด

การเติม @unique ตอนฐานข้อมูลยังว่างคือการแก้ไฟล์บรรทัดเดียวแล้วรัน migration
 การเติม @unique ตอนมีข้อมูลจริงแล้วคือการต้องไล่หาแถวซ้ำ ตัดสินใจว่าจะลบแถวไหน ติดตามผู้ใช้ที่ได้รับผลกระทบ แล้วค่อยรัน migration
 ตอนนี้ฐานข้อมูลยังว่าง นี่คือช่วงเวลาที่ถูกที่สุดและกำลังหมดลง

38 สิ่งที่ยังไม่รู้

เอกสารนี้พยายามไม่เดา สิ่งที่ยังไม่มีคำตอบขอเขียนไว้ตรง ๆ

- **ต่อได้แค่ auth** — backend มี 3 procedure จากที่ระบบต้องการทั้งหมดราว 18 ยังไม่รู้ procedure ที่ซับซ้อนกว่านี้ (เช่นการสร้างคำขอยืมที่มีหลายรายการ) จะเจอข้อจำกัดอะไรเพิ่ม
- **ยังไม่ได้ทดสอบตอน session หมดอายุกลางคัน** — ตั้ง TTL ไว้ 8 ชั่วโมง แต่ยังไม่ไดลองว่าหน้าจจะ ทำตัวอย่างไรเมื่อหมดอายุระหว่างใช้งาน
- **ยังไม่ได้ทดสอบบนมือถือและยังไม่ได้ตรวจ accessibility**
- **ยังไม่ได้ลองรัน CI บน GitHub จริง** — สคริปต์ทุกตัวทดสอบแล้วในเครื่อง แต่ตัว workflow เองยังไม่เคย ผ่าน runner ยังไม่รู้ว่ไบนารีของเครื่องมือสร้างสัญญา มีรุ่นที่ตรงกับ ubuntu-latest หรือไม่

ภาค 8

ภาคผนวก

ภาคผนวก ก — ทุกไฟล์ที่ spike แก้

ตารางนี้คือคำตอบครบถ้วนของคำถาม “spike แก้อะไรจากโค้ดจริงบ้าง” นับจาก diff ระหว่าง frontend/ กับ spike-option-b/frontend-real/

ตารางที่ 13: ไฟล์ที่แก้ 13 ไฟล์ — ตัวเลขคือบรรทัดที่เพิ่ม/ลบ

ไฟล์	+	-	แก้อะไร
package.json	5	3	zod 3→4.4.3, resolvers 3→5.2.2, เพิ่ม @trpc/client
vite.config.ts	7	8	พอร์ต 5173→5275, ตัด proxy
src/types/domain.ts	18	12	User/Role/CreditBand ดึงจากสัญญา
src/types/env.d.ts	2	0	เพิ่ม VITE_TRPC_URL
src/lib/error-messages.ts	30	1	รหัสตรงกับ backend + แยกกรณีนี้ต่อไม่ติด
src/app/app.tsx	28	18	hydrate() → <SessionBootstrap>
src/features/auth/auth.store.ts	19	38	ตัด loginAs และ hydrate ที่
src/features/auth/use-me.ts	15	6	เรียก trpc.auth.me
src/features/auth/login-page.tsx	34	24	เรียกเชิฟเวอร์จริง บทบาทมาจากคำตอบ
...login-method-ku.tsx	24	6	async + ปุ่มมีสถานะ + ที่แสดงข้อผิดพลาด
...login-method-local.tsx	14	8	async + ปุ่มมีสถานะ
src/components/layout/sidebar.tsx	8	3	logout เป็นการเรียกเชิฟเวอร์
src/i18n/locales/{th,en}.json	4	2	เพิ่มคำว่า “กำลังเข้าสู่ระบบ”

ตารางที่ 14: ไฟล์ใหม่ 4 ไฟล์ และไฟล์ที่ลบ 1 ไฟล์

ไฟล์	บรรทัด	คืออะไร
src/lib/trpc.ts	66	ตัวเรียก tRPC + ตัวแยกประเภทข้อผิดพลาด
src/generated/server.ts	56	สัญญา — สร้างโดยเครื่องมือ ห้ามแก้ไข
src/features/auth/use-login.ts	28	mutation สำหรับล็อกอิน
src/features/auth/use-logout.ts	29	mutation สำหรับออกจากระบบ
src/features/auth/mock-auth.ts	-70	ลบทั้งไฟล์

ตัวเลขรวมที่ควรจำ

แก้ 13 ไฟล์ (เพิ่ม 208 บรรทัด ลบ 129) เพิ่มใหม่ 4 ไฟล์ (179 บรรทัด) ลบทิ้ง 1 ไฟล์ (70 บรรทัด) เทียบ กับ ขนาด ของ frontend/ src/ ทั้งหมด แล้ว นี่ คือส่วน น้อย มาก — และ ไม่มี ไฟล์ ใน กลุ่ม components/ui/, features/admin/ หรือ app/router.tsx ถูกแตะเลยแม้แต่ไฟล์เดียว

ภาคผนวก ข — วิธีรัน spike เพื่อดูด้วยตัวเอง

```
# 1. database (shared with the option A spike)
docker compose -f spike-login/docker-compose.yml up -d

# 2. backend on :3002
cd spike-option-b/backend && npm ci && cp .env.example .env
npx prisma generate && npm run dev

# 3. the team's real frontend, wired, on :5275
cd spike-option-b/frontend-real && npm ci && npm run dev
```

เปิด <http://localhost:5275> แล้วใช้บัญชีในตารางที่ 15 รหัสผ่านทั้งหมดถูก hash ด้วย scrypt เก็บในฐานข้อมูลจริง

ตารางที่ 15: บัญชีทดสอบที่ seed.ts สร้างไว้

ช่องที่ใช้	กรอก	รหัสผ่าน	บทบาทที่ได้
อีเมล KU	natthawut.s@ku.th	borrower1234	borrower
อีเมล KU	orawan.p@ku.th	supervisor1234	supervisor
อีเมล KU	lowcredit.s@ku.th	lowcredit1234	borrower เครดิต 20
บัญชีภายใน	test_staff	staff1234	staff
บัญชีภายใน	test_admin	admin1234	admin

สิ่งที่ควรลองกดมี 4 อย่างที่เห็นผลชัดที่สุด

1. ล็อกอินด้วย orawan.p@ku.th — ต้องไปโผล่หน้าอนุมัติของหัวหน้า ไม่ใช่หน้าผู้ยืม (จุดที่ 1)
2. ล็อกอินแล้วกด F5 — ต้องยังอยู่ในระบบ (สถานะที่ 3)
3. เปิดเครื่องมือนักพัฒนา พิมพ์ document.cookie — ต้องไม่เห็น ulms_session (จุดที่ 2)
4. ปิด backend แล้วลองล็อกอิน — ต้องขึ้นข้อความว่าเชื่อมต่อไม่ได้ ไม่ใช่ว่ารหัสผ่านผิด (จุดที่ 12)

ภาคผนวก ค — ผลการทดสอบ 28 ข้อ

ชั้นข้อมูล 13 ข้อ

รันด้วย `node spike-option-b/frontend-real/e2e-check.mjs` ใช้ @trpc/client ตัวเดียวกับที่แอปใช้ จึงครอบคลุมรูปแบบการส่งข้อมูลจริง และวิธีที่ข้อผิดพลาดจากเซิร์ฟเวอร์กลายเป็น TRPCClientError ผังหน้า

เว็บ

```
PASS auth.me without a session → NOT_AUTHENTICATED 401
PASS auth.login local test_staff → role staff
PASS contract field id is a number, not a string
PASS contract field maxBorrowDays present
PASS auth.me after login → same user
PASS auth.login wrong password → INVALID_CREDENTIALS 401
PASS auth.login non-KU email → NOT_KU_EMAIL 400
PASS auth.login identifier > 200 chars → BAD_REQUEST
PASS auth.login KU email of an academic → role supervisor
PASS low-credit student → D3 from the DB
PASS maxBorrowDays for D3 comes from the DB, not CREDIT_BANDS
PASS auth.logout → ok
PASS auth.me after logout → NOT_AUTHENTICATED
```

ชั้นหน้าจอ 15 ข้อ

รันด้วย `node spike-option-b/browser-check/ui-check.mjs` เปิด Firefox จริง กรอกฟอร์มจริง

```
PASS visiting / with no session lands on /login
PASS wrong password shows the mapped Thai message
PASS and stays on /login
PASS staff login lands on the staff dashboard
PASS browser stored the session cookie
PASS and it is httpOnly (JS cannot read it)
PASS document.cookie really cannot see it
PASS reload keeps the session (auth.me bootstrap)
PASS staff visiting /admin is bounced back to /staff
PASS logout returns to /login
PASS session cookie is gone
PASS and /staff is no longer reachable
PASS academic signing in with a KU email lands on SUPERVISOR home
PASS no console errors during the whole run
```

สามข้อในรายการบนที่ทดสอบด้วยวิธีอื่นไม่ได้เลย

คุกกี้เป็น **httpOnly** จริง — ต้องมี JavaScript ที่รันอยู่ในหน้าเว็บถึงจะลองอ่านได้ สคริปต์ยิง HTTP เฉย ๆ ตรวจสอบไม่ได้

รีเฟรชแล้วไม่หลุด — พิสูจน์ว่ากลไก `session` ทำงานถูก

RBAC ยึดจริง — พิมพ์ URL ตรง ๆ แล้วถูกดึงกลับ

ภาคผนวก ง — ที่มาของข้อเท็จจริงทุกข้อ

ถ้าไฟล์เหล่านี้เปลี่ยน เนื้อความในเอกสารต้องตามไปแก้

รูปที่วาดด้วย TikZ เป็นภาพอธิบาย · รูปที่เป็นภาพถ่ายหน้าจอเป็นผลการทดสอบจริง
เอกสารนี้สร้างจาก docs/spike-gap.tex ด้วย latexmk spike-gap.tex

ตารางที่ 16: ข้อเท็จจริงในเอกสารนี้และแหล่งที่มา อ้างถึงบรรทัด ณ วันที่ 10 ส.ค. 2569

ข้อเท็จจริง	ไฟล์:บรรทัด
หน้าล็อกอินกำหนดบทบาทตายตัว	frontend/src/features/auth/login-page.tsx:41
บทบาทเก็บใน localStorage	frontend/src/features/auth/auth.store.ts:36, 46
ตารางรหัสผ่านทดสอบ	frontend/src/features/auth/mock-auth.ts:62
รหัสผ่านทดสอบใน README	frontend/README.md:14- -18
User.id เป็น string	frontend/src/types/domain.ts:10
departmentId ไม่ยอมรับค่าว่าง	frontend/src/types/domain.ts:15
zod รุ่น 3	frontend/package.json:40
ตาราง CREDIT_BANDS	frontend/src/constants/index.ts:44- -49
รหัส INVALID_DOMAIN	frontend/src/lib/error-messages.ts:11
proxy ชั่ว :3000	frontend/vite.config.ts
backend ไม่มี CORS	backend/src/main.ts ทั้งไฟล์ 7 บรรทัด
Email ไม่มี @unique	backend/prisma/schema.prisma:22
UserID ไม่มี @unique	backend/prisma/schema.prisma:24
ไม่มีตาราง session	backend/prisma/schema.prisma ทั้งไฟล์ 33 model
MaxBorrowDate ผูกกับคู่	backend/prisma/schema.prisma:232
CreditTierName ไม่บังคับ	backend/prisma/schema.prisma model CreditTier
RoleInfo เป็นตาราง	backend/prisma/schema.prisma model RoleInfo
สถานะหน้าจอ 8/26	frontend/src/features/**/*-page.tsx
diff ทุกไฟล์	spike-option-b/frontend-real/
ผลทดสอบชั้นข้อมูล	spike-option-b/frontend-real/e2e-check.mjs
ผลทดสอบชั้นหน้าจอ	spike-option-b/browser-check/ui-check.mjs
ภาพหน้าจอ	spike-option-b/browser-check/shots/